

Python Programming and Data Analysis in the Age of AI

A beginner-friendly introduction for undergraduate students

Summer Semester Teaching Materials

1 How To Use This Document

This document introduces Python as a first programming language. It is written for students who may not have programmed before. The goal is not only to learn Python syntax, but also to learn the ideas behind programming: **variables**, functions, conditions, loops, data structures, files, data analysis, and **visualization**.

The examples are intended to be typed, changed, and run. Programming is learned by experimenting. When you see a code block, try to predict what it will do before running it.

In the age of **AI**, programming is not only about memorizing syntax. **AI** tools can help explain error messages, generate examples, and suggest code. But you still need the fundamentals so that you can judge whether code is correct, adapt it to your own problem, and debug it when it fails.

1.1 Course Goals

By the end of this material, students should be able to:

- Explain what programming is and how Python executes code.
- Use **variables**, expressions, functions, conditions, and loops.
- Work with lists, **dictionaries**, strings, and files.
- Write small programs that solve practical problems.
- Read simple **datasets** and compute summaries.
- Create basic plots with `matplotlib` and `pandas`.
- Use **AI** tools responsibly as a learning and productivity assistant.

2 1. What Is Programming?

Programming is the process of giving precise instructions to a computer. A program is a sequence of instructions that the computer can execute.

A computer does not “understand” the world the way a person does. It follows instructions exactly. That is both powerful and frustrating:

- Powerful because computers can repeat tasks quickly and consistently.
- Frustrating because small mistakes in instructions can cause **errors** or wrong results.

2.1 A First Program

```
print("Hello, world!")
```

This program displays text on the screen. In Python, `print()` is a function. A function is a reusable command that performs a task.

2.2 How Python Runs Code

Python reads and runs code from top to bottom.

```
print("First")  
print("Second")  
print("Third")
```

Output:

```
First  
Second  
Third
```

The order matters.

```
print("Before calculation")  
print(10 + 5)  
print("After calculation")
```

3 2. Python As A Programming Language

Python is popular because it is readable, flexible, and widely used in data science, web development, automation, scientific computing, artificial intelligence, and education.

3.1 Why Python For Beginners?

Python has relatively simple syntax:

```
age = 20  
name = "Alex"  
  
print(name)  
print(age)
```

The same idea in some other languages often requires more structure.

JavaScript:

```
let age = 20;
let name = "Alex";

console.log(name);
console.log(age);
```

C#:

```
int age = 20;
string name = "Alex";

Console.WriteLine(name);
Console.WriteLine(age);
```

Python usually requires less punctuation. For beginners, that makes it easier to focus on programming concepts.

3.2 Python, JavaScript, And C#: A Quick Comparison

Concept	Python	JavaScript	C#
Common use	Data, automation, AI, web backends	Web pages, apps, servers	Enterprise apps, games, Windows, web
Variable declaration	<code>x = 5</code>	<code>let x = 5;</code>	<code>int x = 5;</code>
Print output	<code>print(x)</code>	<code>console.log(x)</code>	<code>Console.WriteLine(x);</code>
Blocks of code	Indentation	Curly braces {}	Curly braces {}
Type style	Dynamic	Dynamic	Mostly static
Beginner readability	Very high	Medium	Medium to high, but more formal

Dynamic typing means Python figures out the type of a variable while the program runs. Static typing means the programmer usually declares types in advance.

4 3. Values, Types, And Expressions

A value is a piece of data. Examples:

- 42
- 3.14
- "hello"
- True
- False

Every value has a type. A type describes what kind of data a value is.

4.1 Common Python Types

Type	Meaning	Example
<code>int</code>	Whole number	<code>42</code>
<code>float</code>	Decimal number	<code>3.14</code>
<code>str</code>	Text string	<code>"hello"</code>
<code>bool</code>	Boolean true/false	<code>True</code>
<code>list</code>	Ordered collection	<code>[1, 2, 3]</code>
<code>dict</code>	Key-value collection	<code>{"name": "Alex"}</code>

You can check a value's type with `type()`.

```
print(type(42))
print(type(3.14))
print(type("hello"))
print(type(True))
```

4.2 Expressions

An expression is code that produces a value.

```
print(2 + 3)
print(10 - 4)
print(6 * 7)
print(20 / 5)
```

Python supports common arithmetic operators:

Operator	Meaning	Example	Result
<code>+</code>	Addition	<code>2 + 3</code>	<code>5</code>
<code>-</code>	Subtraction	<code>10 - 4</code>	<code>6</code>
<code>*</code>	Multiplication	<code>6 * 7</code>	<code>42</code>
<code>/</code>	Division	<code>20 / 5</code>	<code>4.0</code>
<code>//</code>	Integer division	<code>20 // 6</code>	<code>3</code>
<code>%</code>	Remainder	<code>20 % 6</code>	<code>2</code>
<code>**</code>	Power	<code>2 ** 3</code>	<code>8</code>

```
print(20 // 6)
print(20 % 6)
print(2 ** 3)
```

4.3 Strings

A string is text.

```
first_name = "Ada"
last_name = "Lovelace"

full_name = first_name + " " + last_name
print(full_name)
```

Python strings can use either single quotes or double quotes.

```
message1 = "Python is fun"
message2 = 'Python is useful'

print(message1)
print(message2)
```

Use f-strings to insert values into text.

f-string stands for “formatted string literal”. It is a Python string prefixed with **f** that allows expressions inside **{}** to be evaluated and inserted into the string.

```
name = "Maya"
score = 92

print(f"{name} earned a score of {score}.")
```

5 4. Variables

A variable is a name that refers to a value. **Variables** let us store information and reuse it later.

```
temperature = 72
print(temperature)
```

You can update a variable:

```
temperature = 72
temperature = 75
```

```
print(temperature)
```

The second assignment replaces the first value.

5.1 Variable Names

Good variable names make code easier to understand.

```
# Good
student_count = 28
average_score = 86.5

# Poor
x = 28
y = 86.5
```

Python naming conventions:

- Use lowercase words separated by underscores: `average_score`.
- Choose names that explain meaning.
- Do not start variable names with numbers.
- Avoid Python reserved words such as `if`, `for`, `class`, and `def`.

5.2 Assignment Is Not Algebra

In mathematics, $x = x + 1$ may look strange. In programming, it means:

1. Take the current value of `x`.
2. Add 1.
3. Store the result back into `x`.

```
x = 10
x = x + 1
print(x)
```

Python also supports a shorter version:

```
x = 10
x += 1
print(x)
```

6 5. Input And Output

Programs often need input from users and output for users.

6.1 Output With `print()`

```
name = "Sam"
age = 19

print("Name:", name)
print("Age:", age)
```

6.2 Input With `input()`

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```

The `input()` function always returns a string.

```
age_text = input("How old are you? ")
print(type(age_text))
```

If you want a number, convert the input.

```
age_text = input("How old are you? ")
age = int(age_text)

print(f"Next year you will be {age + 1}.")
```

Common conversion functions:

Function	Converts To	Example
<code>int()</code>	Whole number	<code>int("42")</code>
<code>float()</code>	Decimal number	<code>float("3.14")</code>
<code>str()</code>	String	<code>str(42)</code>
<code>bool()</code>	Boolean	<code>bool(1)</code>

7 6. Conditions: Making Decisions

Programs become useful when they can make decisions.

```
score = 85

if score >= 60:
    print("Pass")
```

```
else:
    print("Fail")
```

The `if` statement checks a condition. If the condition is true, Python runs the indented code below it.

7.1 Comparison Operators

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to
<code>></code>	Greater than
<code>>=</code>	Greater than or equal to

```
age = 20

print(age == 20)
print(age >= 18)
print(age < 18)
```

7.2 `if`, `elif`, And `else`

Use `elif` when there are multiple possible conditions.

```
score = 87

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(grade)
```


7.3 Boolean Logic

Use `and`, `or`, and `not` to combine conditions.

```
age = 21
has_id = True

if age >= 21 and has_id:
    print("Entry allowed")
else:
    print("Entry denied")
```

```
is_weekend = False
is_holiday = True

if is_weekend or is_holiday:
    print("No class today")
else:
    print("Class today")
```

```
is_raining = False

if not is_raining:
    print("Go for a walk")
```

7.4 Python Versus JavaScript And C# Conditions

Python:

```
if score >= 60:
    print("Pass")
else:
    print("Fail")
```

JavaScript:

```
if (score >= 60) {
    console.log("Pass");
} else {
    console.log("Fail");
}
```

C#:

```

if (score >= 60)
{
    Console.WriteLine("Pass");
}
else
{
    Console.WriteLine("Fail");
}

```

Python uses indentation to define code blocks. JavaScript and C# use curly braces.

8 7. Loops: Repeating Work

A loop repeats code.

There are two main loop types in Python:

- **for** loops: repeat over a sequence of items.
- **while** loops: repeat while a condition is true.

8.1 for Loops

```

for number in [1, 2, 3, 4, 5]:
    print(number)

```

The variable `number` takes each value from the **list** one at a time.

8.2 range()

The `range()` function creates a sequence of numbers.

```

for i in range(5):
    print(i)

```

This prints 0 through 4. In Python, counting often starts at zero.

```

for i in range(1, 6):
    print(i)

```

This prints 1 through 5.

```

for i in range(0, 10, 2):
    print(i)

```

This prints every second number from 0 up to, but not including, 10.

8.3 Looping Over Strings

```
word = "Python"

for letter in word:
    print(letter)
```

8.4 Looping Over Lists

```
scores = [88, 92, 79, 95]

for score in scores:
    print(score)
```

8.5 while Loops

A `while` loop repeats while a condition is true.

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Be careful: if the condition never becomes false, the loop runs forever.

```
# Do not run this unless you know how to stop it.
# count = 1
# while count <= 5:
#     print(count)
```

The problem is that `count` never changes.

8.6 break And continue

Use `break` to stop a loop early.

```
for number in range(1, 10):
    if number == 5:
        break
    print(number)
```

Use `continue` to skip to the next loop iteration.

```
for number in range(1, 6):  
    if number == 3:  
        continue  
    print(number)
```

8.7 Loop Example: Average Score

```
scores = [88, 92, 79, 95]  
  
total = 0  
  
for score in scores:  
    total += score  
  
average = total / len(scores)  
print(average)
```

This example introduces `len()`, which returns the number of items in a collection.

9 8. Functions

A function is a named block of reusable code.

Python already includes many built-in functions:

- `print()`
- `len()`
- `type()`
- `int()`
- `float()`
- `str()`

You can also define your own functions.

9.1 Defining A Function

```
def greet():  
    print("Hello!")  
  
greet()
```

The keyword `def` starts a function definition.

9.2 Parameters

Parameters are inputs to a function.

A parameter is a variable name used in a function definition. It tells the function what kind of input it expects.

An argument is the actual value you pass to the function when you call it.

Example:

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Aisha")  
greet("Ben")
```

9.3 Return Values

A function can return a result.

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)  
print(result)
```

`print()` displays something. `return` sends a value back to the place where the function was called.

9.4 Function Example: Letter Grade

```
def letter_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    elif score >= 60:  
        return "D"  
    else:  
        return "F"  
  
print(letter_grade(95))  
print(letter_grade(72))
```

9.5 Why Functions Matter

Functions help us:

- Avoid repeating code.

- Give names to ideas.
- Break large problems into smaller pieces.
- Test one part of a program at a time.

9.6 Python, JavaScript, And C# Functions

Python:

```
def add(a, b):
    return a + b
```

JavaScript:

```
function add(a, b) {
    return a + b;
}
```

C#:

```
int Add(int a, int b)
{
    return a + b;
}
```

C# normally requires parameter and return types. Python and JavaScript usually do not.

10 9. Lists

A **list** stores multiple values in order.

```
scores = [88, 92, 79, 95]
print(scores)
```

10.1 Indexing

Use an index to access one item. Python indexes start at 0.

```
scores = [88, 92, 79, 95]

print(scores[0])
print(scores[1])
```

Negative indexes count from the end.

```
scores = [88, 92, 79, 95]
```

```
print(scores[-1])
print(scores[-2])
```

10.2 Changing List Items

```
scores = [88, 92, 79, 95]
scores[2] = 81

print(scores)
```

10.3 Adding And Removing Items

```
students = ["Ana", "Bo", "Chen"]

students.append("Dina")
print(students)

students.remove("Bo")
print(students)
```

10.4 Useful List Functions And Methods

```
scores = [88, 92, 79, 95]

print(len(scores))
print(sum(scores))
print(min(scores))
print(max(scores))
```

A method is a function attached to an object. For example, `scores.append(100)` calls the `append()` method on the **list** `scores`.

```
scores = [88, 92, 79, 95]

scores.sort()
print(scores)
```

10.5 Slicing

Slicing extracts part of a **list**.

```
scores = [88, 92, 79, 95, 84]

print(scores[0:3])
```

```
print(scores[2:])
print(scores[:2])
```

`scores[0:3]` includes index 0, 1, and 2, but not index 3.

10.6 List Comprehensions

A **list** comprehension is a compact way to create a new list.

```
numbers = [1, 2, 3, 4, 5]
squares = []

for number in numbers:
    squares.append(number ** 2)

print(squares)
```

The same idea with a **list** comprehension:

```
numbers = [1, 2, 3, 4, 5]
squares = [number ** 2 for number in numbers]

print(squares)
```

Filtering with a **list** comprehension:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = [number for number in numbers if number % 2 == 0]

print(even_numbers)
```

11 10. Dictionaries

A dictionary stores key-value pairs.

```
student = {
    "name": "Maya",
    "major": "Economics",
    "year": 2
}

print(student)
```

Use a key to access a value.


```
print(student["name"])
print(student["major"])
```

11.1 Updating Dictionaries

```
student = {
    "name": "Maya",
    "major": "Economics",
    "year": 2
}

student["year"] = 3
student["gpa"] = 3.7

print(student)
```

11.2 Looping Over Dictionaries

```
student = {
    "name": "Maya",
    "major": "Economics",
    "year": 2
}

for key in student:
    print(key, student[key])
```

Another common style:

```
for key, value in student.items():
    print(key, value)
```

11.3 Example: Counting Words

```
sentence = "data science uses data to understand data"
words = sentence.split()

counts = {}

for word in words:
    if word in counts:
        counts[word] += 1
    else:
```

```
counts[word] = 1

print(counts)
```

This is a small but important data analysis idea: counting how often values appear.

12 11. Strings In More Detail

Strings are sequences of characters.

```
text = "Python programming"

print(text[0])
print(text[7])
print(text[0:6])
```

12.1 Common String Methods

```
text = " Python Programming "

print(text.lower())
print(text.upper())
print(text.strip())
print(text.replace("Programming", "Data Analysis"))
```

12.2 Splitting And Joining

```
sentence = "Python is useful for data analysis"
words = sentence.split()

print(words)
```

```
words = ["Python", "is", "useful"]
sentence = " ".join(words)

print(sentence)
```

12.3 String Example: Cleaning Data

Real data is often messy.

```
raw_major = " computer science "
clean_major = raw_major.strip().title()
```

```
print(clean_major)
```

The `strip()` method removes extra spaces. The `title()` method capitalizes words.

13 12. Errors And Debugging

Errors are normal. Debugging is the process of finding and fixing problems in code.

13.1 Syntax Errors

A syntax error means Python cannot understand the code structure.

```
# Missing closing parenthesis  
# print("Hello"
```

13.2 Runtime Errors

A runtime error happens while the program is running.

```
# Division by zero  
# print(10 / 0)
```

13.3 Logic Errors

A logic error means the program runs, but the result is wrong.

```
scores = [80, 90, 100]  
  
# Incorrect average because the denominator should be len(scores)  
average = sum(scores) / 2  
print(average)
```

Logic **errors** can be the hardest because Python may not show an error message.

13.4 A Debugging Checklist

When code fails:

1. Read the error message from bottom to top.
2. Find the file name and line number.
3. Check spelling, punctuation, and indentation.
4. Print intermediate values.
5. Test one small part at a time.
6. Ask an **AI** tool to explain the error, but verify the answer.

Example:

```

numbers = [10, 20, 30]

for number in numbers:
    print("Current number:", number)
    print("Doubled:", number * 2)

```

14 13. Files

Programs often read data from files and write results to files.

14.1 Reading A Text File

Suppose a file named `notes.txt` contains:

```

Python is readable.
Python is powerful.
Practice matters.

```

Read it with:

```

with open("notes.txt", "r", encoding="utf-8") as file:
    text = file.read()

print(text)

```

The `with` statement automatically closes the file when the block is done.

14.2 Reading Line By Line

```

with open("notes.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())

```

14.3 Writing A Text File

```

with open("output.txt", "w", encoding="utf-8") as file:
    file.write("This file was created by Python.\n")
    file.write("One line at a time.\n")

```

Mode meanings:

Mode	Meaning
"r"	Read
"w"	Write, replacing existing content

Mode	Meaning
"a"	Append to the end

14.4 Reading CSV Files With The Standard Library

CSV stands for comma-separated values. It is a common format for simple **datasets**.

Example file `students.csv`:

```
name,major,score
Ana,Biology,88
Bo,History,92
Chen,Computer Science,79
Dina,Economics,95
```

```
import csv

with open("students.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for row in reader:
        print(row["name"], row["score"])
```

The `csv.DictReader` reads each row as a dictionary.

15 14. Modules And Packages

A module is a file that contains Python code. A package is a collection of modules.

Python includes many built-in modules.

15.1 Importing A Module

```
import math

print(math.sqrt(16))
print(math.pi)
```

You can import specific functions:

```
from math import sqrt

print(sqrt(25))
```

15.2 Common Built-In Modules

Module	Use
<code>math</code>	Mathematical functions
<code>random</code>	Random numbers and choices
<code>datetime</code>	Dates and times
<code>csv</code>	CSV files
<code>json</code>	JSON data
<code>statistics</code>	Mean, median, and related functions

Example with `random`:

```
import random

choices = ["rock", "paper", "scissors"]
computer_choice = random.choice(choices)

print(computer_choice)
```

Example with `statistics`:

```
import statistics

scores = [88, 92, 79, 95]

print(statistics.mean(scores))
print(statistics.median(scores))
```

15.3 Installing External Packages

Many useful Python tools are external packages. They must be installed before use.

Common data analysis packages:

- `numpy`: numerical computing.
- `pandas`: tables and data frames.
- `matplotlib`: plotting.
- `seaborn`: statistical plots.
- `scikit-learn`: machine learning.

Installation often uses `pip`:

```
pip install pandas matplotlib seaborn
```

In a Jupyter notebook, students sometimes use:

```
# !pip install pandas matplotlib seaborn
```

16 15. Data Analysis Foundations

Data analysis usually means:

1. Ask a question.
2. Collect or load data.
3. Clean and organize data.
4. Summarize patterns.
5. Visualize results.
6. Communicate conclusions and uncertainty.

Python is widely used for data analysis because it has excellent libraries.

16.1 A Tiny Dataset In Pure Python

```
students = [  
    {"name": "Ana", "major": "Biology", "score": 88},  
    {"name": "Bo", "major": "History", "score": 92},  
    {"name": "Chen", "major": "Computer Science", "score": 79},  
    {"name": "Dina", "major": "Economics", "score": 95},  
    {"name": "Eli", "major": "Computer Science", "score": 84}  
]
```

Question: What is the average score?

```
total = 0  
  
for student in students:  
    total += student["score"]  
  
average = total / len(students)  
print(average)
```

Question: How many students are in each major?

```
major_counts = {}  
  
for student in students:  
    major = student["major"]  
  
    if major in major_counts:
```

```

        major_counts[major] += 1
    else:
        major_counts[major] = 1

print(major_counts)

```

16.2 Introducing pandas

pandas is the most common Python library for working with table-shaped data.

```

import pandas as pd

students = [
    {"name": "Ana", "major": "Biology", "score": 88},
    {"name": "Bo", "major": "History", "score": 92},
    {"name": "Chen", "major": "Computer Science", "score": 79},
    {"name": "Dina", "major": "Economics", "score": 95},
    {"name": "Eli", "major": "Computer Science", "score": 84}
]

df = pd.DataFrame(students)
print(df)

```

A DataFrame is like a spreadsheet table inside Python.

16.3 Inspecting A DataFrame

```

print(df.head())
print(df.info())
print(df.describe())

```

Common methods:

Method	Meaning
<code>head()</code>	Show first rows
<code>tail()</code>	Show last rows
<code>info()</code>	Show columns, types, and missing values
<code>describe()</code>	Summary statistics
<code>shape</code>	Number of rows and columns
<code>columns</code>	Column names


```
print(df.shape)
print(df.columns)
```

16.4 Selecting Columns

```
print(df["name"])
print(df[["name", "score"]])
```

16.5 Filtering Rows

```
high_scores = df[df["score"] >= 90]
print(high_scores)
```

Filter by text:

```
cs_students = df[df["major"] == "Computer Science"]
print(cs_students)
```

16.6 Creating New Columns

```
df["passed"] = df["score"] >= 60
df["curved_score"] = df["score"] + 3

print(df)
```

16.7 Grouping And Summarizing

```
average_by_major = df.groupby("major")["score"].mean()
print(average_by_major)
```

Count rows by group:

```
count_by_major = df["major"].value_counts()
print(count_by_major)
```

16.8 Reading A CSV With pandas

```
import pandas as pd

df = pd.read_csv("students.csv")
print(df.head())
```

16.9 Saving Results

```
df.to_csv("students_with_results.csv", index=False)
```

17 16. Plotting And Visualization

Visualization helps us see patterns that are hard to understand from tables alone.

17.1 Basic Plot With `matplotlib`

```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5]
temperatures = [70, 72, 71, 75, 78]

plt.plot(days, temperatures)
plt.xlabel("Day")
plt.ylabel("Temperature")
plt.title("Temperature Over Time")
plt.show()
```

17.2 Bar Chart

```
import matplotlib.pyplot as plt

majors = ["Biology", "History", "Computer Science", "Economics"]
counts = [1, 1, 2, 1]

plt.bar(majors, counts)
plt.xlabel("Major")
plt.ylabel("Number of Students")
plt.title("Students By Major")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

17.3 Scatter Plot

```
import matplotlib.pyplot as plt

hours_studied = [2, 3, 4, 5, 6, 7]
exam_scores = [70, 75, 78, 85, 88, 93]

plt.scatter(hours_studied, exam_scores)
plt.xlabel("Hours Studied")
```

```
plt.ylabel("Exam Score")
plt.title("Study Time And Exam Score")
plt.show()
```

17.4 Plotting With pandas

```
import pandas as pd
import matplotlib.pyplot as plt

students = [
    {"name": "Ana", "major": "Biology", "score": 88},
    {"name": "Bo", "major": "History", "score": 92},
    {"name": "Chen", "major": "Computer Science", "score": 79},
    {"name": "Dina", "major": "Economics", "score": 95},
    {"name": "Eli", "major": "Computer Science", "score": 84}
]

df = pd.DataFrame(students)

df["score"].plot(kind="hist", bins=5, title="Distribution Of Scores")
plt.xlabel("Score")
plt.show()
```

17.5 A Better Bar Chart From Grouped Data

```
major_counts = df["major"].value_counts()

major_counts.plot(kind="bar", title="Students By Major")
plt.xlabel("Major")
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

17.6 Visualization Principles

Good charts should:

- Have a clear title.
- Label axes.
- Use appropriate chart types.
- Avoid unnecessary decoration.
- Make comparisons easy.
- Represent data honestly.

Common chart choices:

Goal	Chart Type
Change over time	Line chart
Compare categories	Bar chart
Show relationship between two numbers	Scatter plot
Show distribution	Histogram
Show parts of a whole	Use carefully; often a bar chart is better

18 17. Mini Project: Analyze Course Survey Data

This project combines **variables**, lists, **dictionaries**, functions, loops, **pandas**, and plotting.

18.1 Scenario

Suppose students answer a short survey:

- How many hours per week do you study?
- How confident do you feel in Python from 1 to 5?
- What is your major?

18.2 Create A Dataset

```
import pandas as pd

survey = pd.DataFrame({
    "student": ["S1", "S2", "S3", "S4", "S5", "S6", "S7", "S8"],
    "major": [
        "Biology",
        "Computer Science",
        "Economics",
        "Biology",
        "History",
        "Computer Science",
        "Economics",
        "Biology"
    ],
    "study_hours": [4, 8, 5, 6, 3, 10, 7, 5],
    "confidence": [2, 4, 3, 3, 2, 5, 4, 3]
})

print(survey)
```

18.3 Summarize The Data

```
print(survey.describe())
```

```
print(survey["major"].value_counts())
```

```
print(survey.groupby("major")[["study_hours", "confidence"]].mean())
```

18.4 Ask A Question

Question: Do students who study more hours tend to report higher confidence?

```
import matplotlib.pyplot as plt

plt.scatter(survey["study_hours"], survey["confidence"])
plt.xlabel("Study Hours Per Week")
plt.ylabel("Python Confidence")
plt.title("Study Hours And Python Confidence")
plt.show()
```

18.5 Compute A Correlation

Correlation measures the direction and strength of a linear relationship between two numeric variables.

```
correlation = survey["study_hours"].corr(survey["confidence"])
print(correlation)
```

Correlation values are usually between -1 and 1:

- Near 1: strong positive relationship.
- Near 0: little linear relationship.
- Near -1: strong negative relationship.

Important: correlation does not prove causation.

18.6 Write A Short Conclusion

Example conclusion:

In this small survey **dataset**, students who reported more study hours also tended to report higher Python confidence. The sample is very small, so this pattern should be treated as exploratory rather than conclusive.

19 18. Object-Oriented Programming Preview

Python supports **object-oriented programming**. This means organizing code around objects that combine data and behavior.

Students do not need to master this immediately, but they should recognize the idea.

19.1 Objects And Methods

In Python, many things are objects.

```
name = "python"
print(name.upper())
```

Here, `name` is a string object, and `.upper()` is a method.

19.2 Classes

A class is a blueprint for creating objects.

```
class Student:
    def __init__(self, name, major):
        self.name = name
        self.major = major

    def introduce(self):
        print(f"My name is {self.name} and I study {self.major}.")

student1 = Student("Maya", "Economics")
student1.introduce()
```

In this example:

- `Student` is a class.
- `student1` is an object.
- `name` and `major` are attributes.
- `introduce()` is a method.

19.3 Python, JavaScript, And C# Classes

Python:

```
class Student:
    def __init__(self, name):
        self.name = name
```

JavaScript:

```
class Student {
    constructor(name) {
        this.name = name;
    }
}
```

C#:

```
class Student
{
    public string Name;

    public Student(string name)
    {
        Name = name;
    }
}
```

C# uses explicit types and access modifiers such as `public`. Python is less formal, which makes small examples shorter.

20 19. Programming In The Age Of AI

AI tools can make programming more accessible, but they do not remove the need to understand code.

20.1 Useful Ways To Use AI

AI can help with:

- Explaining unfamiliar code.
- Explaining error messages.
- Generating small examples.
- Suggesting test cases.
- Translating code between languages.
- Improving comments and documentation.
- Brainstorming approaches to a problem.

20.2 Risky Ways To Use AI

Be careful when AI:

- Produces code you cannot explain.
- Makes up library functions or package names.
- Ignores edge cases.
- Gives a correct-looking but wrong answer.
- Uses outdated syntax.

- Solves an assignment in a way that bypasses learning.

20.3 A Good AI Prompt For Learning

Weak prompt:

```
Write my Python homework.
```

Better prompt:

```
I am learning Python loops. I need to count how many numbers in a list are greater than 10. Please explain the idea first, then show a small example, then give me a practice problem without the answer.
```

20.4 A Good AI Prompt For Debugging

```
I am a beginner in Python. This code gives an error. Please explain what the error means, point to the likely line, and suggest the smallest fix. Do not rewrite the whole program unless necessary.
```

```
<paste code here>
```

```
<paste full error message here>
```

20.5 Always Verify AI-Generated Code

When AI gives you code:

1. Read it line by line.
2. Run it on a small example.
3. Check whether the output makes sense.
4. Try an edge case.
5. Explain the code in your own words.

21 20. Practice Exercises

These exercises are designed to reinforce the fundamentals.

21.1 Variables And Expressions

1. Create **variables** for your name, age, and major. Print a sentence using an f-string.
2. Convert a temperature from Fahrenheit to Celsius using the formula $(F - 32) * 5 / 9$.
3. Calculate the total cost of three items with tax.

21.2 Conditions

1. Write a program that checks whether a number is positive, negative, or zero.
2. Write a program that assigns a letter grade from a numeric score.
3. Write a program that checks whether a person is eligible for a discount based on age or student status.

21.3 Loops

1. Print the numbers from 1 to 20.
2. Print only the even numbers from 1 to 20.
3. Given a **list** of scores, calculate the average.
4. Count how many words in a **list** have more than five letters.

21.4 Functions

1. Write a function `square(number)` that returns the square of a number.
2. Write a function `is_even(number)` that returns `True` if the number is even.
3. Write a function `calculate_average(numbers)` that returns the average of a **list**.
4. Write a function `letter_grade(score)` that returns "A", "B", "C", "D", or "F".

21.5 Lists And Dictionaries

1. Create a **list** of five cities. Print the first and last city.
2. Add a new city to the **list** and remove one city.
3. Create a dictionary for a book with title, author, and year.
4. Given a sentence, count how often each word appears.

21.6 Data Analysis

1. Create a DataFrame with at least five rows about movies, including title, year, genre, and rating.
2. Find the average rating.
3. Count how many movies are in each genre.
4. **Plot** a **bar chart** of average rating by genre.

22 21. Suggested Teaching Schedule

This document can be adapted to different course lengths. For a summer semester, one possible sequence is:

Week	Topics	Suggested Activity
1	What programming is, Python setup, values, variables	First programs and f-strings
2	Conditions and loops	Grade calculator and number games
3	Functions, lists, dictionaries	Build small reusable tools
4	Strings, files, debugging	Text cleaning and word counts
5	Modules, packages, pandas basics	Load and inspect CSV data
6	Grouping, summaries, plotting	Survey data mini project

Week	Topics	Suggested Activity
7	AI-assisted programming, verification, ethics	Debugging with AI and code review
8	Final project	Student-designed data analysis

23 22. Final Project Ideas

Students can choose a small **dataset** and answer a question using Python.

Possible projects:

- Analyze course survey results.
- Explore movie ratings by genre and year.
- Analyze sports **statistics**.
- Study weather trends.
- Compare prices of products.
- Analyze public health or demographic data.
- Visualize music or streaming data.

A simple final project should include:

1. A research question.
2. A **dataset**.
3. **Data cleaning** or preparation.
4. At least two summary **statistics**.
5. At least one **visualization**.
6. A short written interpretation.
7. A note about limitations.

24 23. Key Takeaways

Programming is about precise problem solving. Python is a readable language that lets beginners learn core ideas without too much syntax overhead.

The most important beginner concepts are:

- **Variables** store values.
- Types describe kinds of values.
- Conditions make decisions.
- Loops repeat work.
- Functions organize reusable logic.
- Lists and **dictionaries** store collections.
- Files and packages connect programs to the outside world.
- Data analysis uses code to summarize, visualize, and interpret data.
- **AI** can help, but understanding and verification remain essential.

25 Appendix A: Python Syntax Cheat Sheet

25.1 Variables

```
name = "Alex"  
age = 20
```

25.2 Printing

```
print("Hello")  
print(f"{name} is {age} years old")
```

25.3 Conditions

```
if age >= 18:  
    print("Adult")  
else:  
    print("Not adult")
```

25.4 Loops

```
for i in range(5):  
    print(i)
```

```
count = 0  
while count < 5:  
    print(count)  
    count += 1
```

25.5 Functions

```
def add(a, b):  
    return a + b
```

25.6 Lists

```
numbers = [1, 2, 3]  
numbers.append(4)  
print(numbers[0])
```

25.7 Dictionaries

```
student = {"name": "Alex", "score": 90}  
print(student["name"])
```

25.8 Pandas

```
import pandas as pd  
  
df = pd.read_csv("data.csv")  
print(df.head())  
print(df.describe())
```

25.9 Matplotlib

```
import matplotlib.pyplot as plt  
  
plt.plot([1, 2, 3], [10, 20, 15])  
plt.xlabel("X")  
plt.ylabel("Y")  
plt.title("Simple Plot")  
plt.show()
```

26 Appendix B: Common Beginner Mistakes

26.1 Confusing = And ==

```
x = 5      # assignment  
print(x == 5) # comparison
```

26.2 Forgetting Indentation

```
if x > 0:  
    print("Positive")
```

26.3 Mixing Strings And Numbers

```
age_text = "20"  
age = int(age_text)  
print(age + 1)
```

26.4 Off-By-One Errors

```
for i in range(1, 6):  
    print(i)
```

This prints 1 through 5, not 1 through 6.

26.5 Changing A List While Looping Over It

This can cause confusing behavior:

```
# Avoid this pattern as a beginner.  
# numbers = [1, 2, 3, 4]  
# for number in numbers:  
#     if number % 2 == 0:  
#         numbers.remove(number)
```

Use a new **list** instead:

```
numbers = [1, 2, 3, 4]  
odd_numbers = []  
  
for number in numbers:  
    if number % 2 != 0:  
        odd_numbers.append(number)  
  
print(odd_numbers)
```

27 Appendix C: Small Reference Dataset

Students can use this small CSV data by saving it as `students.csv`.

```
name,major,study_hours,score  
Ana,Biology,4,88  
Bo,History,5,92  
Chen,Computer Science,7,79  
Dina,Economics,6,95  
Eli,Computer Science,8,84  
Fatima,Biology,3,81  
Gabe,History,2,76  
Hana,Economics,9,91
```

Example analysis:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("students.csv")

print(df.head())
print(df.describe())
print(df.groupby("major")["score"].mean())

df.plot(kind="scatter", x="study_hours", y="score", title="Study Hours And Score")
plt.show()
```